

CITED REFERENCES AND FURTHER READING:

- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §2.2.[1]
- Floater, M.S., and Hormann, K. 2006+, “Barycentric Rational Interpolation with No Poles and High Rates of Approximation,” at <http://www.in.tu-clausthal.de/fileadmin/homes/techreports/ifi0606hormann.pdf>. [2]
- Berrut, J.-P., and Trefethen, L.N. 2004, “Barycentric Lagrange Interpolation,” *SIAM Review*, vol. 46, pp. 501–517. [3]
- Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall), §6.2.
- Cuyt, A., and Wuytack, L. 1987, *Nonlinear Methods in Numerical Analysis* (Amsterdam: North-Holland), Chapter 3.

3.5 Coefficients of the Interpolating Polynomial

Occasionally you may wish to know not the value of the interpolating polynomial that passes through a (small!) number of points, but the coefficients of that polynomial. A valid use of the coefficients might be, for example, to compute simultaneous interpolated values of the function and of several of its derivatives (see §5.1), or to convolve a segment of the tabulated function with some other function, where the moments of that other function (i.e., its convolution with powers of x) are known analytically.

Please be certain, however, that the coefficients are what you need. Generally the coefficients of the interpolating polynomial can be determined much less accurately than its value at a desired abscissa. Therefore, it is not a good idea to determine the coefficients only for use in calculating interpolating values. Values thus calculated will not pass exactly through the tabulated points, for example, while values computed by the routines in §3.1 – §3.3 will pass exactly through such points.

Also, you should not mistake the interpolating polynomial (and its coefficients) for its cousin, the *best-fit* polynomial through a data set. Fitting is a *smoothing* process, since the number of fitted coefficients is typically much less than the number of data points. Therefore, fitted coefficients can be accurately and stably determined even in the presence of statistical errors in the tabulated values. (See §14.9.) Interpolation, where the number of coefficients and number of tabulated points are equal, takes the tabulated values as perfect. If they in fact contain statistical errors, these can be magnified into oscillations of the interpolating polynomial in between the tabulated points.

As before, we take the tabulated points to be $y_i \equiv y(x_i)$. If the interpolating polynomial is written as

$$y = c_0 + c_1x + c_2x^2 + \cdots + c_{N-1}x^{N-1} \quad (3.5.1)$$

then the c_i 's are required to satisfy the linear equation

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{N-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{N-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{N-1} & x_{N-1}^2 & \cdots & x_{N-1}^{N-1} \end{bmatrix} \cdot \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{N-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{bmatrix} \quad (3.5.2)$$

This is a *Vandermonde matrix*, as described in §2.8. One could in principle solve equation (3.5.2) by standard techniques for linear equations generally (§2.3); however, the special method that was derived in §2.8 is more efficient by a large factor, of order N , so it is much better.

Remember that Vandermonde systems can be quite ill-conditioned. In such a case, *no* numerical method is going to give a very accurate answer. Such cases do not, please note, imply any difficulty in finding interpolated *values* by the methods of §3.2, but only difficulty in finding *coefficients*.

Like the routine in §2.8, the following is due to G.B. Rybicki.

polcoef.h

```
void polcoe(VecDoub_I &x, VecDoub_I &y, VecDoub_0 &cof)
Given arrays x[0..n-1] and y[0..n-1] containing a tabulated function  $y_i = f(x_i)$ , this routine
returns an array of coefficients cof[0..n-1], such that  $y_i = \sum_{j=0}^{n-1} \text{cof}_j x_i^j$ .
{
    Int k,j,i,n=x.size();
    Doub phi,ff,b;
    VecDoub s(n);
    for (i=0;i<n;i++) s[i]=cof[i]=0.0;
    s[n-1] = -x[0];
    for (i=1;i<n;i++) {
        for (j=n-1-i;j<n-1;j++)
            s[j] -= x[i]*s[j+1];
        s[n-1] -= x[i];
    }
    for (j=0;j<n;j++) {
        phi=n;
        for (k=n-1;k>0;k--)
            phi=k*s[k]+x[j]*phi;
        ff=y[j]/phi;
        b=1.0;
        for (k=n-1;k>=0;k--) {
            cof[k] += b*ff;
            b=s[k]+x[j]*b;
        }
    }
}
```

Coefficients s_i of the master polynomial $P(x)$ are found by recurrence.

The quantity $\text{phi} = \prod_{j \neq k} (x_j - x_k)$ is found as a derivative of $P(x_j)$.

Coefficients of polynomials in each term of the Lagrange formula are found by synthetic division of $P(x)$ by $(x - x_j)$. The solution c_k is accumulated.

3.5.1 Another Method

Another technique is to make use of the function value interpolation routine already given (polint; §3.2). If we interpolate (or extrapolate) to find the value of the interpolating polynomial at $x = 0$, then this value will evidently be c_0 . Now we can subtract c_0 from the y_i 's and divide each by its corresponding x_i . Throwing out one point (the one with smallest x_i is a good candidate), we can repeat the procedure to find c_1 , and so on.

It is not instantly obvious that this procedure is stable, but we have generally found it to be somewhat *more* stable than the routine immediately preceding. This method is of order N^3 , while the preceding one was of order N^2 . You will find, however, that neither works very well for large N , because of the intrinsic ill-condition of the Vandermonde problem. In single precision, N up to 8 or 10 is satisfactory; about double this in double precision.

void polcof(VecDoub_I &xa, VecDoub_I &ya, VecDoub_O &cof)

polcoef.h

Given arrays xa[0..n-1] and ya[0..n-1] containing a tabulated function $ya_i = f(xa_i)$, this routine returns an array of coefficients cof[0..n-1], such that $ya_i = \sum_{j=0}^{n-1} cof_j xa_i^j$.

```
{
    Int k,j,i,n=xa.size();
    Doub xmin;
    VecDoub x(n),y(n);
    for (j=0;j<n;j++) {
        x[j]=xa[j];
        y[j]=ya[j];
    }
    for (j=0;j<n;j++) {
        VecDoub x_t(n-j),y_t(n-j);
        for (k=0;k<n-j;k++) {
            x_t[k]=x[k];
            y_t[k]=y[k];
        }
        Poly_interp interp(x,y,n-j);
        cof[j] = interp.rawinterp(0,0.);
        xmin=1.0e99;
        k = -1;
        for (i=0;i<n-j;i++) {
            if (abs(x[i]) < xmin) {
                xmin=abs(x[i]);
                k=i;
            }
            if (x[i] != 0.0)
                y[i]=(y[i]-cof[j])/x[i];
        }
        for (i=k+1;i<n-j;i++) {
            y[i-1]=y[i];
            x[i-1]=x[i];
        }
    }
}
```

Fill a temporary vector whose size decreases as each coefficient is found.

Extrapolate to $x = 0$.

Find the remaining x_i of smallest absolute value

(meanwhile reducing all the terms)

and eliminate it.

If the point $x = 0$ is not in (or at least close to) the range of the tabulated x_i 's, then the coefficients of the interpolating polynomial will in general become very large. However, the real “information content” of the coefficients is in small differences from the “translation-induced” large values. This is one cause of ill-conditioning, resulting in loss of significance and poorly determined coefficients. In this case, you should consider redefining the origin of the problem, to put $x = 0$ in a sensible place.

Another pathology is that, if too high a degree of interpolation is attempted on a smooth function, the interpolating polynomial will attempt to use its high-degree coefficients, in combinations with large and almost precisely canceling combinations, to match the tabulated values down to the last possible epsilon of accuracy. This effect is the same as the intrinsic tendency of the interpolating polynomial values to oscillate (wildly) between its constrained points and would be present even if the machine's floating precision were infinitely good. The above routines polcoe and polcof have slightly different sensitivities to the pathologies that can occur.

Are you still quite certain that using the *coefficients* is a good idea?

CITED REFERENCES AND FURTHER READING:

Isaacson, E., and Keller, H.B. 1966, *Analysis of Numerical Methods*; reprinted 1994 (New York: Dover), §5.2.